

PROJECT REPORT: Hotel & Room Booking System

Submitted by: Names: Khawar Rahim, Alamzaib, Mudasir Naseem

[Roll no 302-231013,302-231012,302-231032]

Department: BS Software Engineering (BSSE)

Subject: Database Systems

1. Project Overview

1.1 Introduction

The **Hotel & Room Booking System** is a web-based application designed to automate the management of hotel operations. It allows users to register, search for rooms based on availability, and make reservations. For administrators, it provides a dashboard to manage rooms, bookings, and implement dynamic pricing strategies.

1.2 Objectives

- To design a relational database schema that ensures data integrity and minimizes redundancy.
- To implement a 3-Tier Architecture connecting Frontend, Backend API, and Database.
- To demonstrate CRUD (Create, Read, Update, Delete) operations using SQL.
- To handle complex relationships between Users, Rooms, and Bookings.

1.3 Technology Stack

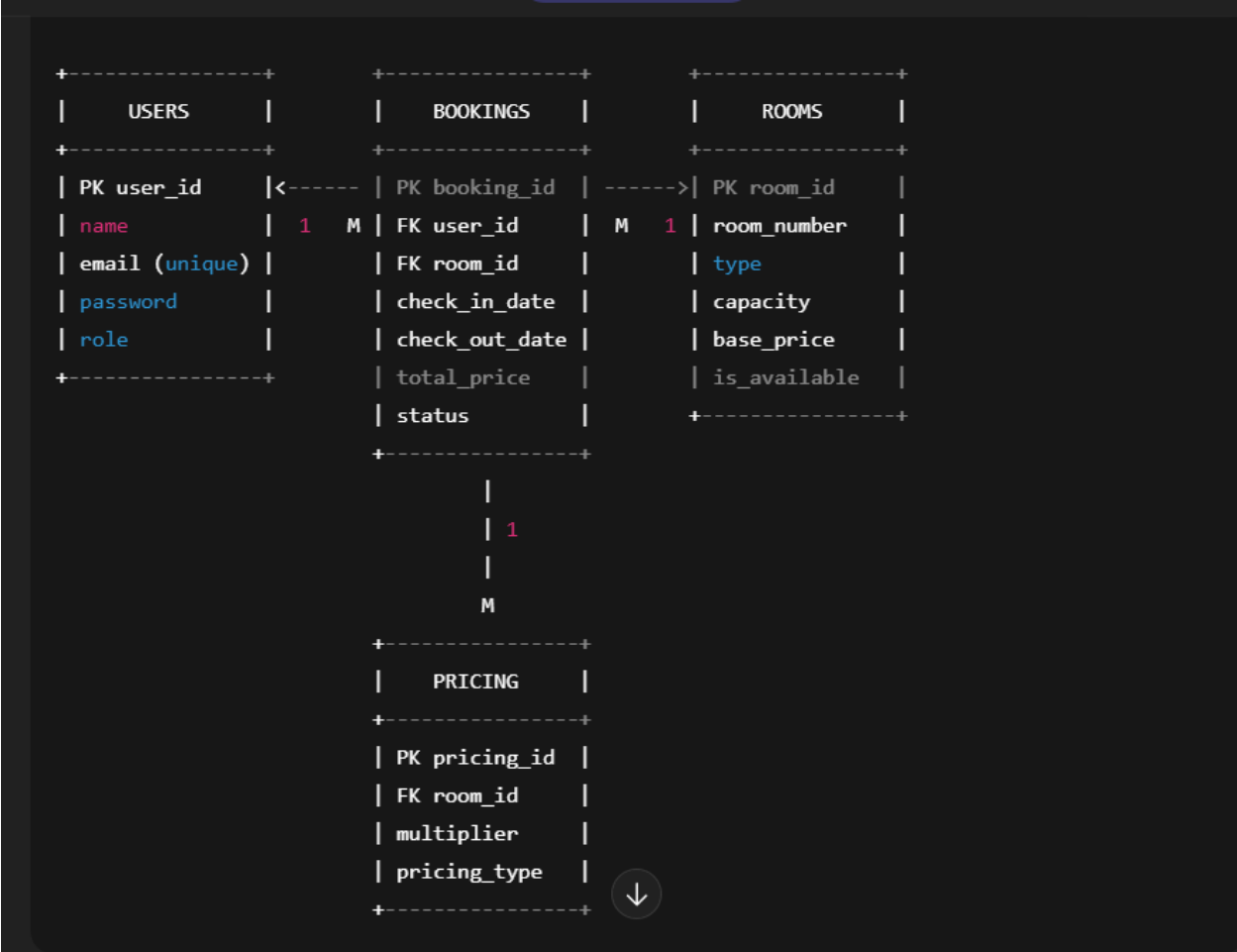
- **Database:** SQLite (Relational Database Management System)
- **Backend:** Node.js & Express.js
- **Frontend:** HTML5, CSS3, JavaScript (Vanilla)
- **Query Language:** SQL (Structured Query Language)

2. Database Design

2.1 Entity Relationship (ER) Diagram

The database is built upon three core entities: **Users**, **Rooms**, and **Bookings**.

- **Users:** Can make multiple bookings (One-to-Many).
- **Rooms:** Can be booked multiple times on different dates (One-to-Many).
- **Bookings:** Acts as the associative entity linking Users and Rooms.
- **Pricing:** Linked to Rooms for seasonal/weekend rates (One-to-Many).



2.2 Database Schema (Table Structures)

Table 1: USERS

Stores customer and admin information.

- **Primary Key:** id
- **Attributes:** name, email (Unique), password (Hashed), role ('user' or 'admin').

Table 2: ROOMS

Stores details of hotel inventory.

- **Primary Key:** id
- **Attributes:** room_number, type, capacity, base_price, is_available.

Table 3: BOOKINGS

Stores reservation transactions. This table enforces relationship integrity.

- **Primary Key:** id
- **Foreign Key:** user_id (References Users table)
- **Foreign Key:** room_id (References Rooms table)
- **Attributes:** check_in, check_out, total_price, status.

Table 4: PRICING

Stores rules for dynamic pricing (e.g., weekends, holidays).

- **Foreign Key:** room_id
- **Attributes:** multiplier (e.g., 1.5x), type (weekend/seasonal).

3. Implementation Details (Key SQL Queries)

3.1 Fetching Bookings with Joins

To display bookings on the Admin Dashboard, we use an `INNER JOIN` to retrieve data from three different tables simultaneously.

SQL

SELECT

```
b.id, b.check_in_date, b.total_price, b.booking_status,  
u.name as user_name, u.email,  
r.room_number, r.room_type
```

FROM bookings b

JOIN users u ON b.user_id = u.id

JOIN rooms r ON b.room_id = r.id

ORDER BY b.check_in_date DESC;

3.2 Adding a New Room (Insert Operation)

SQL

```
INSERT INTO rooms (room_number, room_type, capacity, base_price,
description, is_available)
VALUES (?, ?, ?, ?, ?, 1);
```

3.3 Dynamic Role Management (Update Operation)

Security is managed by checking the user role stored in the database.

SQL

```
UPDATE users SET role = 'admin' WHERE email = ?;
```

4. User Interface & Functionality

4.1 User Registration & Login

Secure authentication system using hashed passwords.

 **HotelBook**

Login to Your Account

Email Address

Password

Login

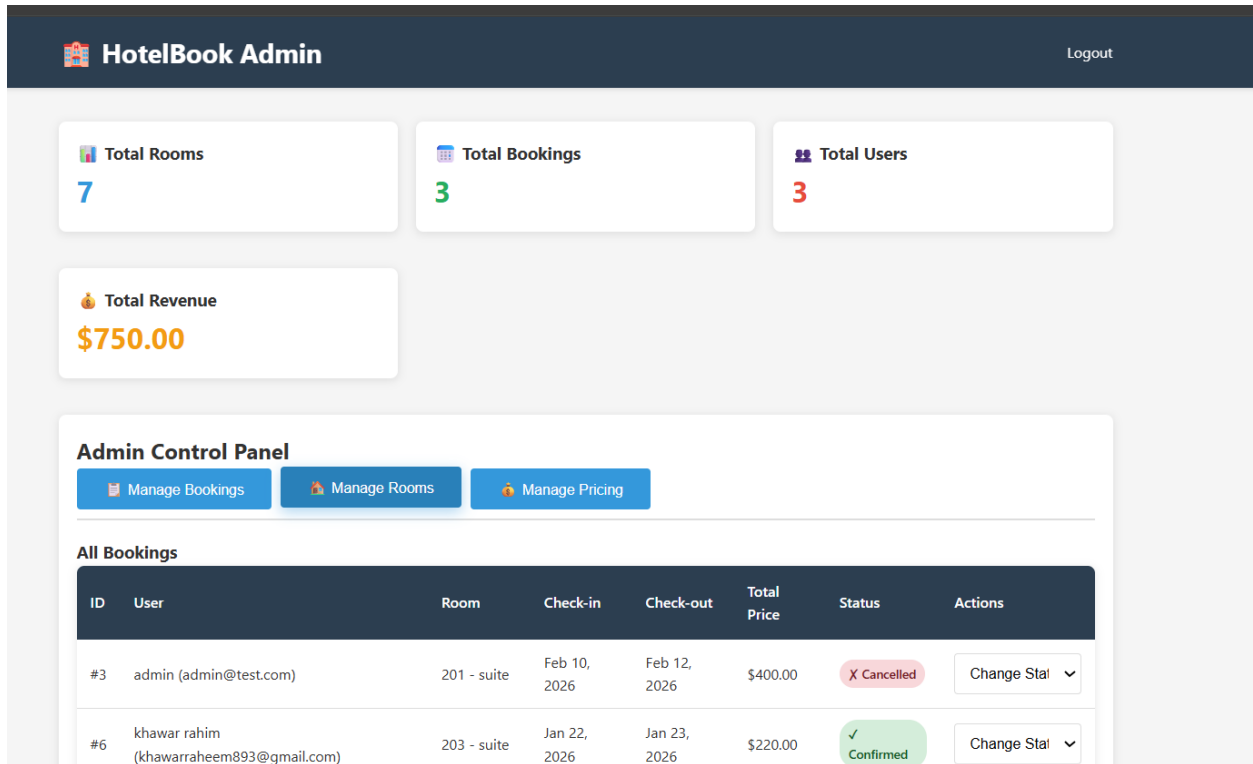
Don't have an account? [Register here](#)

© 2026 Hotel Booking System. All rights reserved.

[Login Page ka screenshot]

4.2 Admin Dashboard

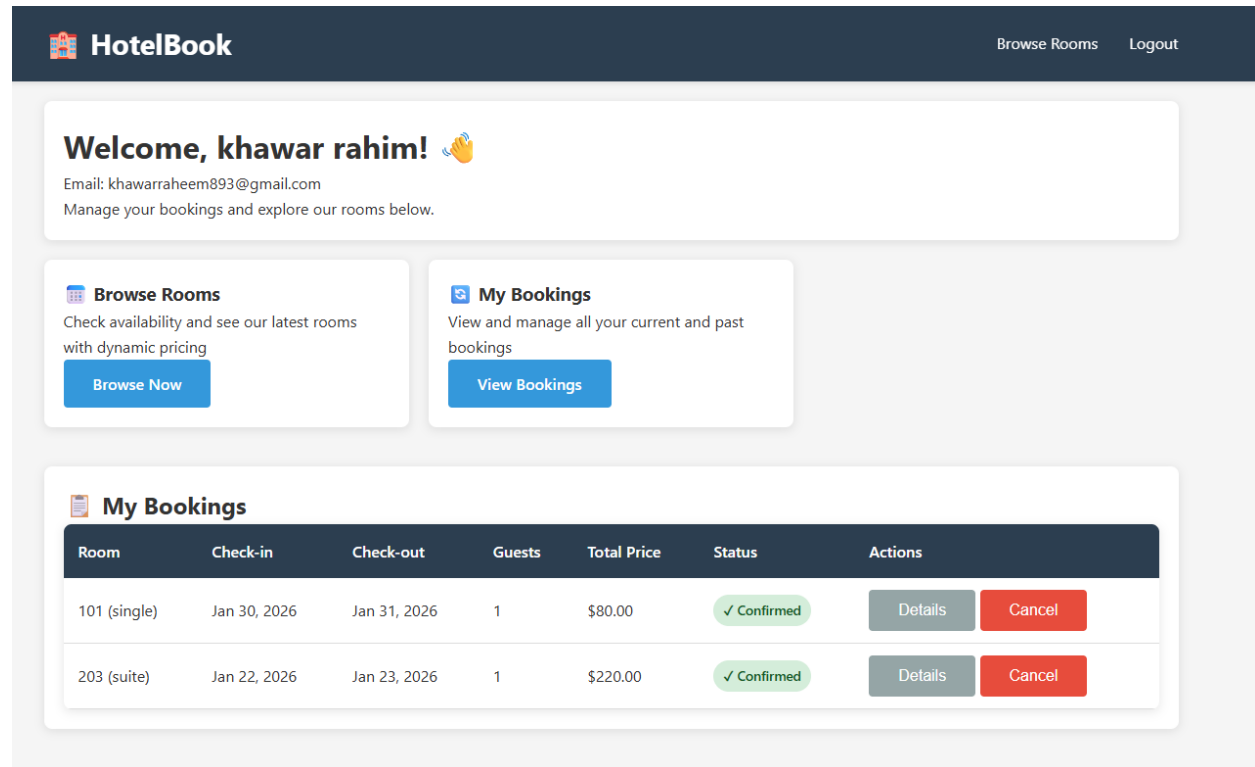
Allows the hotel manager to add rooms and view all active bookings.



[Admin Dashboard screenshot]

4.3 Room Booking Interface

Users can browse rooms and confirm bookings. The system checks availability in the database before confirming.



[Booking Popup/Browse Rooms screenshot]

5. Conclusion

5.1 Conclusion

This project successfully demonstrates the implementation of a database-driven application. By using **Foreign Key Constraints**, data integrity is maintained (e.g., a booking cannot exist without a valid user and room). The use of **Indexing** on email and room IDs ensures fast query performance.